

EXPERIMENT NO: 9

Student Name: Abhi Kumar

UID: 23BCS12907

Branch: BE-CSE

Secon/Group: KRG-1B

Semester: 6th

Date of Performance: 26/02/2026

Subject Name: System Design

Subject Code: 23CSH-314

Aim

To design a scalable and reliable **Ticket Booking System** that enables users to search events, view event details, and book seats securely while supporting **100 Million Daily Active Users**, ensuring **high availability for browsing** and **strong consistency for bookings**.

Objectives

- To design a scalable **Ticket Booking System** for movies, concerts, and live events.
- To understand functional requirements like event search, event details, and seat booking workflow.
- To implement non-functional requirements such as **high availability, strong consistency**, and large-scale user support.
- To design REST APIs, core entities, and optimized database schema for booking operations.
- To create a high-level architecture capable of handling **100 Million Daily Active Users** efficiently.

Procedure

1. Study the problem statement and identify the functional requirements such as event search, event details display, and ticket booking.
2. Analyze non-functional requirements including scalability, high availability, and consistency during seat booking.
3. Identify core entities such as User, Event, Venue, Show, Seat, Booking, and Payment.
4. Design RESTful APIs for searching events, viewing details, selecting seats, making payments, and confirming bookings.
5. Create the database schema with tables and relationships for users, events, shows, seats, bookings, and payments.
6. Design the High-Level Architecture using components like API Gateway, Search Service, Booking Service, Cache, Database, and Notification Service.
7. Implement seat locking and transaction mechanisms to avoid double booking and maintain data consistency.

8. Test the system flow for searching events, selecting seats, successful booking, and handling concurrent requests.

Functional Requirements

1. The system must allow users to search events using filters such as Event Title, City/Location, and Date.
2. The system must display complete event details including description, venue, timing, cast, language, duration, ratings, and seat availability.
3. The system must allow users to select preferred seats from the seat layout.
4. The system must enable users to book tickets by completing the payment process.
5. The system must generate booking confirmation with ticket details after successful payment.
6. The system must allow users to register, log in, and manage their profiles.
7. The system must maintain booking history for users.
8. The system must send notifications through email or SMS for booking confirmation, cancellation, or updates.
9. The system must allow administrators to add, update, or remove events and schedules.
10. The system must track seat status as available, locked, or booked in real time.

Core Entities of the System

- Users
- Event
- Venue
- Show
- ViewHistory
- Seat
- Booking
- Payment

API Design

1. Users :-

- a) POST:- /users/register
- b) POST :- /users/login
- c) GET :- /users/{userID}
- d) GET :- /users/{userID}/bookings -> [Get the booking history]

2. Event Discovery :-

- a) GET :- /events/{filter} -> [Get the list of all the events based on the query param filter]
- b) GET - /events/{eventId} -> [Get the particular event]

3. Seat Managements APIs :-

- a) GET /events/{eventId}/seats → Fetch seat layout and availability
- b) POST /seats/lock → Temporarily lock selected seats
- c) POST /seats/release → Release locked seats (timeout or manual)

4. Booking and Payment APIs :-

- a) POST /bookings → Create a booking (after seat lock)
- b) GET /bookings/{bookingId} → Fetch booking details

5. Notification APIs:-

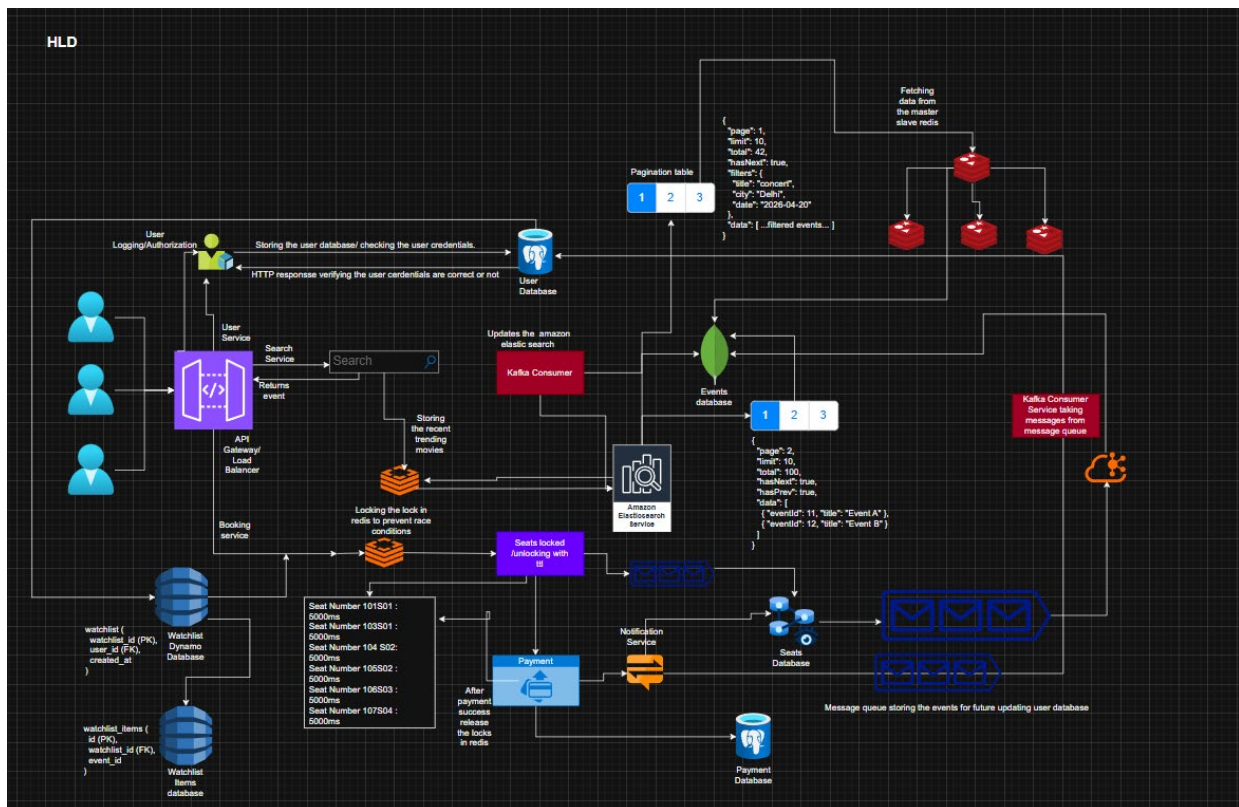
- a) POST /notifications/send → Send booking confirmation (Email/SMS)
- b) POST/notifications/trending → Send the latest information about trending movies.

Non-Functional Requirements

1. System must handle extremely high throughput for concurrent uploads and millions of simultaneous views.
2. High Availability must be prioritized, ensuring no single point of failure.
3. System should follow an Eventual Consistency model for social metrics (view counts, likes) to optimize database write loads.
4. Video delivery latency should be minimized via Edge Caching and a Global CDN.
5. Transcoding pipeline must be horizontally scalable to handle viral upload spikes.

High Level Design (HLD)

The Ticket Booking System follows a microservices-based High Level Design where all client requests first pass through an API Gateway/Load Balancer for routing and traffic management. Core services include User Service for authentication, Search Service integrated with Elasticsearch and Redis for fast event discovery, and Booking Service for seat reservation. Redis is used for temporary seat locking to prevent double booking, while Seats and Payment databases ensure transactional consistency. Kafka enables asynchronous communication for notifications, event updates, and system scalability.



Outcome

- Designed a scalable microservices-based Ticket Booking System capable of handling high user traffic efficiently.
- Implemented fast event search using Elasticsearch and Redis caching for low-latency responses.
- Ensured strong consistency in seat booking through Redis-based seat locking and transactional databases.
- Integrated payment and notification services using Kafka for reliable asynchronous processing.
- Improved system availability, fault tolerance, and overall scalability through distributed architecture.